Unit 14 - Event Driven Programming
Unit 15 - Object Oriented Programming
Unit 22 - Developing Computer Games
Assignment 4 - Testing and Evaluation

Introduction

Scenario / Vocational Context - *As a member of your organisations' software development team you are required to test and evaluate a computer game in .NET C# that you have created.*

Task 1 - Testing *(Unit 14 P5, Unit 15 P5, Unit 22 P5)*

Task description - *You have been asked to thoroughly test your own game using "black box" tests. To do this, you should:*

- *Create a plan*
- *Add tests that test functionality and demonstrate that the program fulfils the design aims and other requirements. This will need at least six (6) tests.*
  You should include tests that:
  - *Makes sure that the key requirements are met.*
    [You wrote these as the first part of the design and implement assignment.]
  - *Test all keyboard input.*
  - *Test functionality within game (such as collision detection, scoring, movement)*
  - *Test game over and restarting.*
- *Complete the test plan and provide a log of your testing by producing screen-prints that show the result of every test in the plan (if possible).*

Create a plan

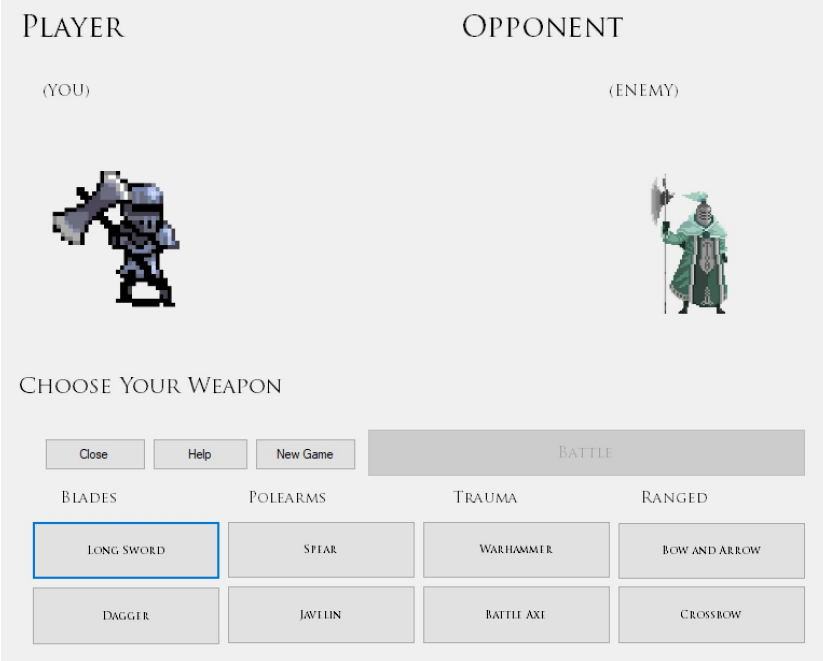
Test Number	Priority	Action	Expected result	Actual Result
1.	High	Does the application open upon starting it?	The application should open when the user presses the "Start" button.	The application carries out the expected result without any sign of errors.
2.	High	Is it possible for the user to select one of the "weapon" buttons?	When the user presses one of the weapons, the application should cancel out the rest of the buttons, indicating that it	The application carries out the expected result without any sign of errors.

			is selected.	
3.	High	Does the “Battle” button unlock when a user chooses a weapon?	When a user presses one of the weapons, the battle button should be ready to be clicked, essentially becoming available (true).	The application carries out the expected result without any sign of errors.
4.	High	Does the battle form from the application open when the “Battle” button is pressed?	The application should open the battle screen when the user presses the “Battle” button.	The application carries out the expected result without any sign of errors.
5.	High	Does the enemy's health reduce when the player clicks “Light Attack”?	When the player presses the “Light Attack” button, the value of both the enemy and player’s health bar should reduce the amount of the selected weapon.	The application carries out the expected result without any sign of errors.
6.	High	Does the enemy's health reduce when the player clicks “Heavy Attack”?	When the player presses the “Heavy Attack” button, the value of both the enemy and player’s health bar should reduce the amount of the selected weapon.	Initially, the application carries out the expected result without any sign of errors. However, the application crashes with the use of most, if not all, weapons when extensively using the button.
7.	High	Does the lose screen appear when the player wins?	If the opponent’s health is reaches zero before the player’s health, the “lose” screen should open.	The application carries out the expected result without any sign of errors.
8.	High	Does the win screen appear when the player wins?	If the opponent’s health is reaches zero before the player’s health, the “win” screen should open.	It was not possible to open the win screen due to the incorrect balancing of the game’s damage and healing system.
9.	Moderate	Does the player’s health increase, with opponent also increasing by half the amount, when the player clicks “heal”?	When the player presses the “Heal” button, the value of both the enemy and player’s health bar should increase, with the player’s health going up by 10 and the enemy by 5.	The application carries out the expected result without any sign of errors. However, due to the functionality of this system, the game is rendered impossible to “win”.
10.	Moderate	Is it possible for the user to click on different weapons when pressing on “New Game” after completing a game previously?	When the player presses the “New Game” button, all the weapon buttons will be set to true, rendering them active to	The application carries out the expected result without any sign of errors.

			an event. Additionally, the “Battle” button should be set to false, rendering it unclickable.	
11.	Moderate	Does the help screen open when the “Help” button is pressed?	When the user presses the “help” button in the main menu, it should open a new window with the information of the game’s instructions and purpose .	The application carries out the expected result without any sign of errors.
12.	Moderate	Is it possible to use both keyboard and mouse input for the event handlers?	The user should be able to navigate through the application via keyboard interaction (arrow keys) or mouse interaction (mouse click).	The application carries out the expected result without any sign of errors.
13.	Low	Does the application close when the “exit” button is pressed?	The application should close upon the user clicking the “exit” button	The application carries out the expected result without any sign of errors.
14.	Low	Are the animations of the characters working?	The sprites within the video game should be animated when launched.	The application carries out the expected result without any sign of errors.

Test Logs

Test No.1



The screenshot above references the test of the application opening when the user starts it, proving that it was successful.

Test No.2



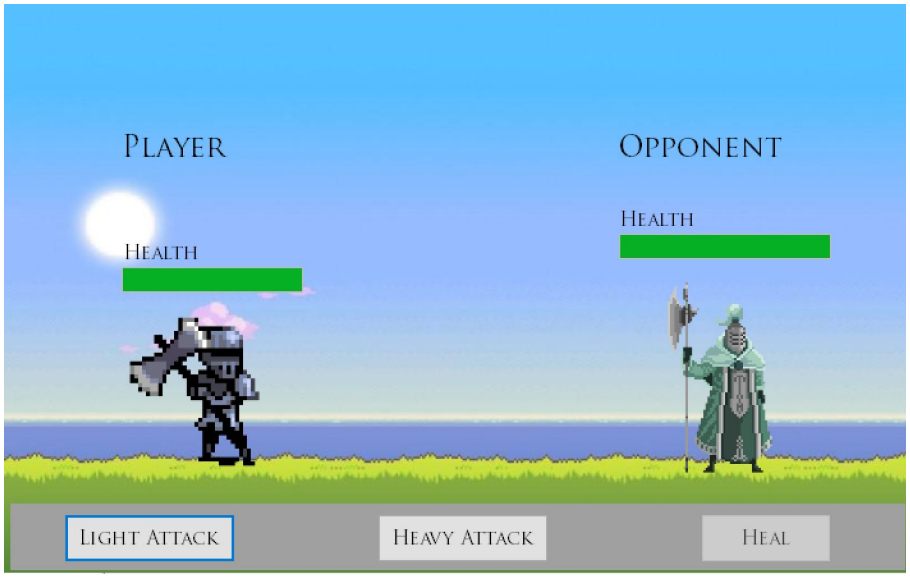
The screenshots above references the test of the user being able to choose the available weapons, proving that it was successful.

Test No.3



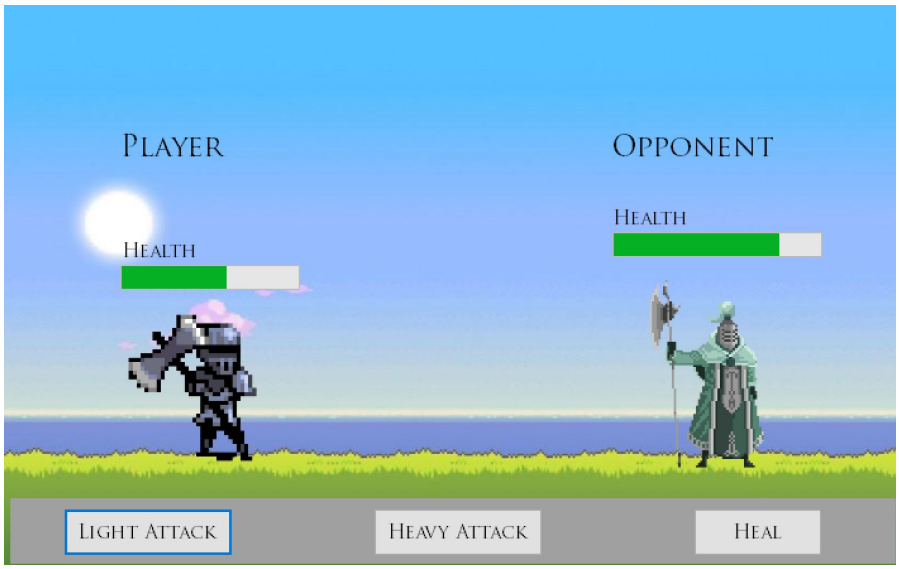
The screenshot above references the test of the battle button activating when a weapon is chosen, proving that it was successful.

Test No.4



The screenshot above references the test of the battle screen opening when the user presses the battle button, proving that it was successful.

Test No.5



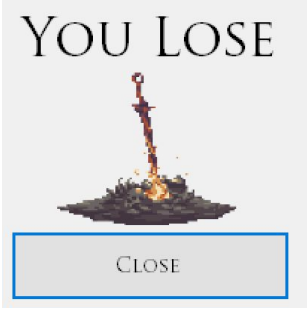
The screenshot above references the test of the “Light Attack” button reducing the progressbar (health) value of both the player and the opponent, proving that it was successful.

Test No.6

```
130             else
131             {
132                 btnHeal.Enabled = true;
133                 prgPlayer.Value -= gamedata.attackingweopon.HeavyDamage * 2;
134             }
135
136
```

This is a screenshot of the attempt of pressing the “Heavy Attack” button pressed by the user. The code shown above is what was responsible for the game crashing. Therefore, the test has failed due to the application not working as intended.

Test No.7

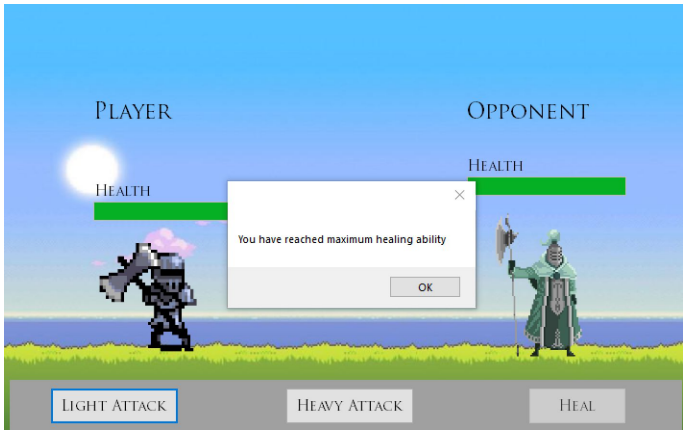


The screenshot above references the test of the “Lose” screen opening when the user’s health is below that of the opponent’s health, proving that it was successful.

Test No.8

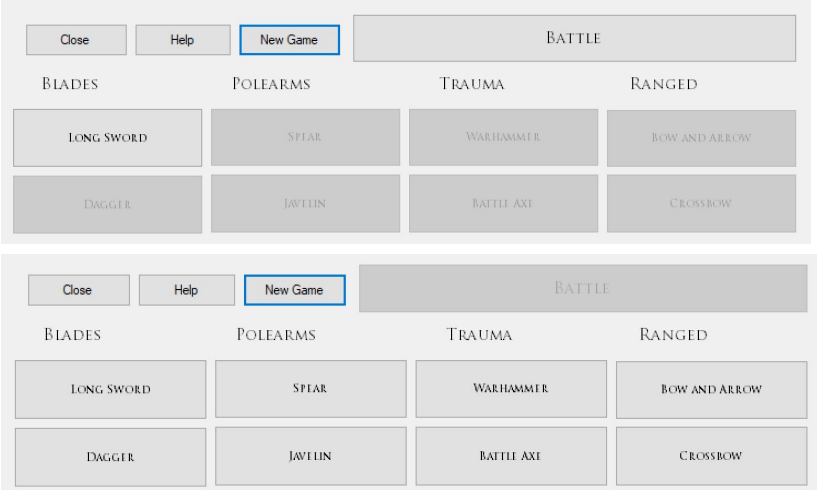
No screenshot is possible to show both keyboard and mouse inputs within the application. However, the test did pass because both keyboard and mouse events do work, which is essential for the application’s ability to function initially.

Test No.9



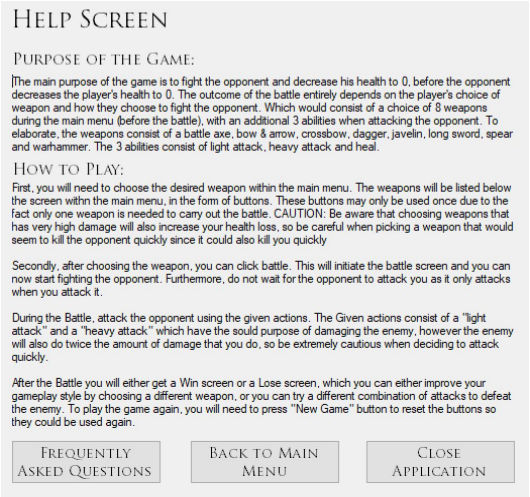
The screenshot above references the test of the heal button restoring health (adding value back to the progress bar) of both the player and the opponent, with a limited cap when the maximum is reached. Proving that the test was successful.

Test No.10



The screenshots above references the test of the “New Game” button restoring all the weapon buttons, essentially making their value from false to true, proving that the test was successful.

Test No.11



The screenshot above references the test of the help screen opening when the user presses the “Help” button, proving that the test was successful.

Test No.12

No screenshot is possible to show both keyboard and mouse inputs within the application. However, the test did pass because both keyboard and mouse events do work, which is essential for the application’s ability to function initially.

Test No.13

Also, this test cannot be provided with a screenshot of the game closing from the close button. Therefore it is not possible to provide evidence with a screenshot. Although, the test was successful due to the fact the programme carried out its intended purpose.

Test No.14

Yet again, due to the nature of this test, no screenshot is possible to show the animations of the sprites used within the game application. However, this again was a successful test because it carried out the expected results.

Task 2 - Analysis of Tests (*Unit 14 M3, Unit 15 M3, Unit 22 M3)*

Task description - *For two tests that failed:*

- *Use a breakpoint to stop the code at the point where you think the program is about to go wrong (show screenshot).*
- *Use Visual Studio's debug tools to show the content of relevant variables (show screenshot).*
- *Step through the code until the program goes 'wrong' (show screenshot at this point)*
- *Explain how this process (i.e using breakpoints, monitoring the content of variables and single stepping) helped you to diagnose the problem.*
- *Provide a screenshot of the game being played that shows that the problem has been corrected.*

For two other tests that failed:

- *Explain why you think the test failed.*
- *How you would correct the problem.*

Test No.6

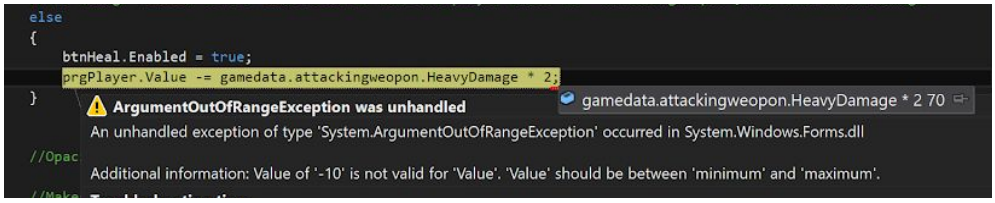
Use a breakpoint to stop the code at the point where you think the program is about to go wrong (show screenshot)

```
130             else
131             {
132                 btnHeal.Enabled = true;
133                 prgPlayer.Value -= gamedata.attackingweopon.HeavyDamage * 2;
134             }
135
```

The screenshot above is the line of code which I was suspicious of causing the game to crash, due to the fact it was under the "Heavy Attack" event handler, and that it determined the value removed by the player's progress bar to the attacking weapon chosen.

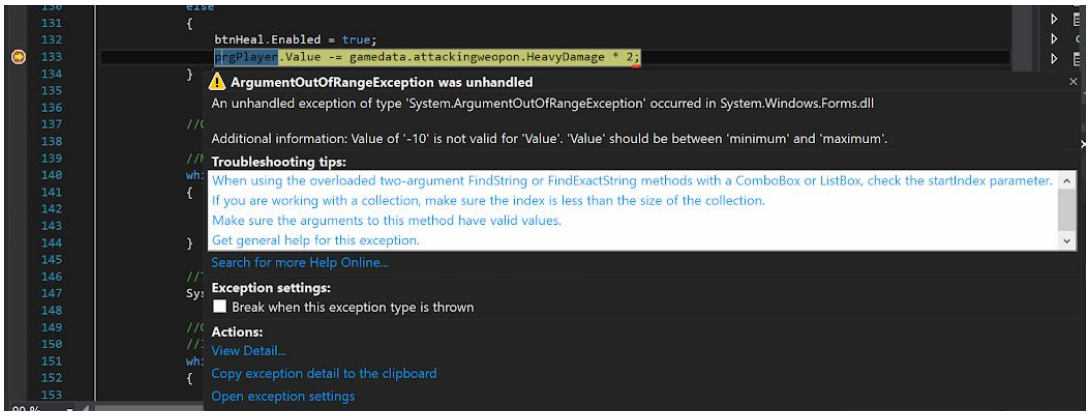
Use Visual Studio's debug tools to show the content of relevant variables (show screenshot)

```
else
{
    btnHeal.Enabled = true;
    prgPlayer.Value -= gamedata.attackingweopon.HeavyDamage * 2;
}
    Argum  prgPlayer.Value 60   was unhandled
    An unhandled exception of type 'System.ArgumentOutOfRangeExceptio
```



The screenshots above is the value of the variables after the application crashing. As seen above, it shows that the value of the player's progress bar (health) is currently at 60. While the value which reduces the value of the said progress bar is at 70.

Step through the code until the program goes 'wrong' (show screenshot at this point)



The screenshot above shows when the program suspectively goes "wrong", which was correct. Due to the fact the value of the progress bar would be at -10, making it impossible for the application to display the said value, causing the programme to crash.

Explain how this process (i.e using breakpoints, monitoring the content of variables and single stepping) helped you to diagnose the problem

To elaborate on the error, while I was testing out the "Heavy Attack" button, I came across a bug which caused the game to crash. However, it was only at certain scenarios where the "Heavy Attack" button would crash the programme. The error message for the crash is shown in the screenshots shown above, with a breakpoint preset to indicate where the error occured.

After reading the error message and finding out why the error has occured, I opened the debugger to inspect the code step by step. With the debugger, I set a breakpoint where the error occured. With this information I could see the value of the player's progress bar was currently at 60, due to it being reduced in gameplay.

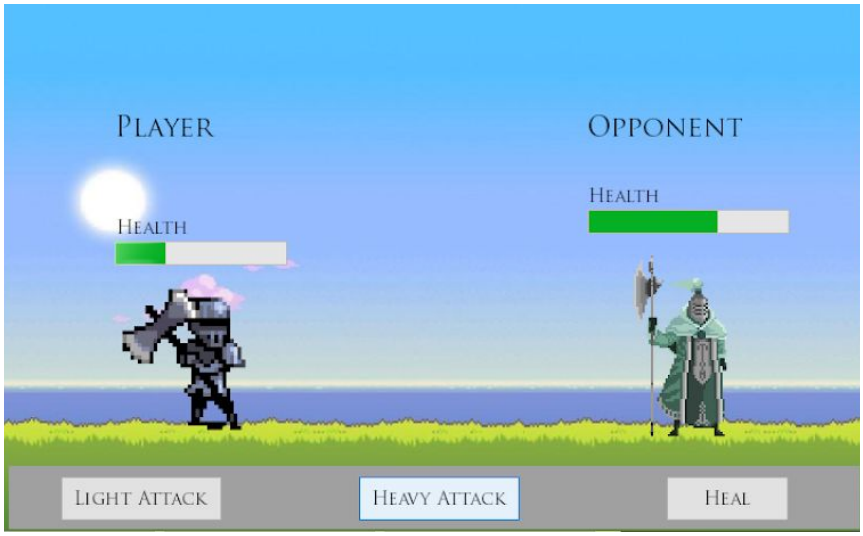
However what caused the crash, as seen in the screenshots above, was the damage (value that reduces the value of the progressbar) from the attacking weapon is 70. Therefore, this would cause the value of the player's health bar (progressbar) to be -10, which is not possible for the application.

Therefore, with that knowledge I decided to overcome the issue with the "Heavy Damage" button. The way this will be carried out is by replacing the damage done by the opponents weapons, which will replace the declaration of "gamedata.attackingweopong.HeavyDamage *2;" to "gamedata.attackingweopong.HeavyDamage + 10;", to reduce the vast reduction in the progress bar value.

With this declaration, the damage that would be done will be minimal compared to the previous method, reducing the chances of the programme crashing drastically. Essentially, I reduced the damage done by the enemy to the player, where it would reduce the chances of the programme crashing due to the progress bar being in the minus. The reason for this number is due to the fact that weapon damage is added by an extra 10 damage, instead of scaling it in multiple of 2s.

Overall, as illustrated on the screenshots shown above, I have found a bug within the application which, when the heavy attack button is pressed, there is a significant chance of the value of the player's progress bar being lower to that of the enemy's weapon damage. Causing the application to crash. The way in which I have fixed it is by only reducing the amount of damage caused by simply reducing the damage done by enemy, with the damage having an additional 10 damage rather than a multiple of 2.

Provide a screenshot of the game being played that shows that the problem has been corrected



After implementing the code, I tested the application to see if the bug was resolved and that the application had an element of error handling and reporting. As shown in the screenshot above, which references the test of the "Heavy Attack" button reducing the progressbar (health) value of both the player and the opponent, proving that it was successful.

After testing the program, I came to the conclusion that the added declarations did indeed function as intended, and that the program can no longer crash from the user adding too much to the progress bar value.

Test No.8

Explain why you think the test failed

The reason that this test has failed wasn't necessarily due to the test itself, but rather the mathematics of the game. To elaborate, the reason that it was impossible to win, was due to the healing giving the player + 10 value and the enemy +5 value and the damage of the enemy being twice the amount of the player.

This in turn creates a mathematical balance within the damage output, making it a consistent stalemate if the player carries on healing. If the player does not heal, the only possible outcome will be to lose the

game to the opponent. Essentially, the damage of the enemy is twice the amount of the player, but the healing of the player is twice the amount of the enemy.

How you would correct the problem

The way which this could be corrected is through the same method I used to fix the programme above, which is to reduce the multiplication of the damage by 2. Which was mathematically incorrect, as I mentioned previously.

Essentially, by removing the multiple of 2, I mathematically corrected the game to the balance of the healing and the damage done. The way I have done this, as mentioned in the previous test I have failed before, was to swap the damage value of the opponent to +10 instead of multiplying by 2.

Task 3 - User Feedback and Possible Improvements *(Unit 22 M4)*

Task description - *Using a table:*

- *Have three of your fellow students play your game. Collect their feedback. This should include more than one strength and possible (non-trivial) improvements.*
- *Add your own thoughts to the table, and identify and explain clearly three suitable, and non-trivial, improvements. Non-trivial in this context means that the improvements should be functional, and not merely changing a colour, for example.*
- *Prioritise the possible improvements, explaining which you think are most important, which are least important and why.*

Have three of your fellow students play your game. Collect their feedback. This should include more than one strength and possible (non-trivial) improvements.

User's Name	Strengths of Your Game	Possible Improvements
(NOTE: These reviews were carried out before the test carried out above)		
Jordan Healy	The animations, though basic, do have a certain appeal. The breathing movements of the sprites are able to make them more personified. The weapon options are thorough enough, and the options easy to follow and understandable. The help menu is incredibly thorough and details all aspects of the game that a player may have difficulty following, although the language used in this help menu is passive, rather than authoritative, making it seem like the writer of the help menu is not confident of	The game, though primarily operational, currently operates at no higher level that the superficial. The game starts correctly, with the application opening and the 2 characters appearing and the option of selecting a weapon to fight with. For the purpose of example, I chose a *'long sword [sic]'* and pressed the *'Battle'* button that leads into the combat. This is where the real problem with the game started, as although the combat (heavy and light) an healing mechanisms worked fine, the logical error behind them means that it's mathematically impossible for the player to win. This is a significant flaw in the game's mechanics. Possible improvements, behind the structure of the game's baseline code, and therefore on a

	the applications usage; some people may prefer this style however.	more auxiliary level that in-depth, is the possible either change of the character in terms of player option, or the change of the player's character, as two-headed axes are not realistic and not used historically. This is a simple matter of appearance, but perhaps changing the sprite to represent the change in weaponry could be done to maximise the game's optimisation. It's not a required change, and the game does not suffer from the lack of it. Beyond that, the only things I would personally say required change are the passive tone of the help menu (words such as 'can' or 'should' or 'could', all words that are passive in tone rather than authoritative), and the dumbing down of the user interface, supplementing the *'Player'* and *'Opponent'* headlines with *'(YOU)'* and *'(ENEMY)'* respectively.
Ryan Da Silva	The animations of the game are extremely well done and look very vivid. They also suit the game very much due to the medieval battle theme. The help menu is also quite detailed and explains how to play the game quite well.	Although the game was functional in the sense that you could play it, it did have two main issues. The first issue comes in the form of the fact that you are unable to actually beat the opponent as they deal double the amount of damage to you and you are unable to out heal them so that you may beat them. The second issue comes in the form of the fact some of the weapons when used cause the damage done by the enemy to go up by a factor of 2 causing the game to crash. There is also a spelling mistake in the help menu, where you have misspelt soul as sould.
Ruben Caldeira	The depth of the help menu is especially useful for someone who is unfamiliar with this type of video game genre. All the questions available in the FAQ menu is also fairly useful. The amount of different weapons available is also interesting, as it adds more diversity onto the game.	During my first playthrough I encountered an error straight away, it occured when I tried to do a 'Heavy Attack' with a Sword, it also occured during my second playthrough with a different weapon; I'd say that fixing this is high priority. The gameplay itself feels quite bland, the player only has 2 choices in general, attack or heal. Adding a third ability would make the game more interesting, or perhaps changing abilities depending on what weapon the player chose.

Add your own thoughts to the table, and identify and explain clearly three suitable, and non-trivial improvements.

User's Review	Personal Opinions	Possible Improvements
Jordan Healy	My personal opinion to this review is that it is a well thought out review, criticising the important sections of the game to the very details and inaccuracies. Although the strengths of this user review was reassuring, the critiques were much more helpful in the improvements for the application. All in all, I believe that this is a well made review and is well criticising of the application.	The possible improvements that I can create to my programme, thanks to this review, is that, since it refers to the mathematical issue with the application, there is a high priority improvement needed in order to fix this issue as soon as possible, so the game is playable. Another issue which is lower in priority, is the sprites of the game changing the weapons depending on the weapon chosen. Which will be an improvement that will be implemented after the bug fixed of the application. Lastly, are the lower priority issues which include grammatical and spelling mistakes. Which include incorrect spelling of longsword, and the use of passive language.
Ryan Da Silva	My personal opinion to this review is that is compliments the visuals and the details of the application. While the critique both focuses on the major issues that was found and the much smaller issues, which required a significant amount of attention to detail to find.	The possible improvements that I can create to my programme, thanks to this review, is that, again, the game was unbeatable due to the mathematical problems, which was pointed out previously. Additionally, the user mentioned how the application crashed due to the weapon having a damage multiplier of 2. However, this issue is less prominent, but need further testing and improvements to fully reduce. Lastly, are again low priority issues which relates to spelling mistakes in the help menu, referring to "sould" instead of "should".
Ruben Calderia	My personal opinion to this review is that it is very short, but direct to the positives and significantly more direct towards the prominent issues within the application.	The possible improvements that I can create to my programme, thanks to this review, is that it refers to the heavy attack issue, which has been resolved to a degree in the previous tests. However, it needs further testing and more refined improvements to fully remove the issue. Lastly, the lower priority issues which need to be improved is the lack of special abilities, making it feel "quite bland". Therefore, to increase the personality of the game, I would need to improve on the combat abilities.
Overall, overlooking all the minor issues found with the game, all user reviews have pointed out the significant, mathematically incorrect aspect of the video game, which causes the programme to crash. While some of the reviews mentioned it was still mathematically impossible to win, and that the application would still crash, means that the previous attempts of correcting the issue was inert. Therefore creating a high priority bug which needs to be patched to a much higher degree then the tests carried out previously. Other low priority improvements mostly consist of grammatical/spelling		

mistakes and other improvements to the UI, including the game's "personality" and "blandness". Which needs to be patched as soon as the other priority issues are resolved.

Prioritise the possible improvements, explaining which you think are most important, which are least important and why

User's Review	Priority	Importance	Justification for the importance of improvements
Jordan Healy	Very High	I believe that the importance of this review ranges from very high to moderate, with the context of the different possible improvements.	The reason that I feel the importance of this review is of high is due to the fact it focuses on the most prominent issue. Which is the mathematical issue which impact the core gameplay of the application. Furthermore, the reason it also consists of moderate priority improvements is due to the fact that grammatical/spelling errors were mentioned. Also, other smaller details which make the weapons more historically accurate, and the mentioning of sprites having different weapons depending on the ones chosen.
Ryan Da Silva	High	I believe that the importance of this review ranges from high to low, again due to the context of the possible improvements.	The reason I feel that the importance consists of high priority is due to player being unable to beat the opponent, as mentioned from the previous user review. However, the reason I also think it has low priority issues is due to the smaller improvements which consist of a spelling mistake and an issue which is already going through the process of being resolved within the programme.
Ruben Caldeira	Moderate	I believe that the importance of this review ranges from moderate to low, with the context of the different possible improvements.	The reason I feel that the importance of this review is less than others due to the fact it focuses on the issues which have been tested and resolved. However, these issues still exist within the programme, hence why the priority is moderate instead of high.
Overall, due to the high priority issue which is still prominent within the video game, consisting of the mathematical problems of the user not being able to beat the opponent, and the game crashing when the value of the progress bar goes below 0. In order to overcome this ussel, while exclusively considering the highest priority issues, there needs to be a significant change in how the health is displayed to the user, rather than the unreliable use of progress bars.			

Task 4 - Evaluation *(Unit 14 D2, Unit 15 D2)*

Task description - *Evaluation. (write around 2 pages)*

- *Discuss how your game meets the original requirements. [You wrote these as the first part of the design and implement assignment. Include discussion of strengths where it works well or exceeds the original requirements].*
- *Discuss at least two strengths and two weaknesses of C# in providing the user interface and in handling the various events you wanted to use to trigger code to run.*
- *Discuss at least two advantages and two disadvantages of using classes, inheritance and polymorphism when creating your game.*

Discuss how your game meets the original requirements

The way in which my game has met the original requirements is that it is essentially an event driven programme, which relies on the use of object oriented paradigm in order to create classes which inherit and polymorphism valuable data from each other.

To elaborate, the original requirements were to create a video game which is focused on both event driven and object oriented paradigm, hence why the C# language was used. The game should essentially be a game which has a system which would be fun to play, have event driven functions and object oriented aspects heavily implemented into the game, using inheritance and polymorphism.

Overall, the game I have created has met the original requirements because it is a game which has the basic functionality to make it fun to play. While also featuring heavy use of event handlers, such as buttons, and heavy use of classes for the weapons within the game, which have inheritance and polymorphism.

Discuss at least two strengths and two weaknesses of C# in providing the user interface and in handling the various events you wanted to use to trigger code to run

The use of C# in terms of providing a user interface is very useful, especially with third part applications such as visual studio, instead of raw code. Essentially, visual basic features are the ability to have premade functions for buttons, list boxes, text boxes and other event handlers. It also has the ability to be able to just drag and drop GUI objects into place for things text boxes, list boxes and buttons for example. It also has a large libraries of examples for you to see and use, which can also be disadvantages however.

One strength that C# provides, in terms of handling various events that I want to use the trigger code to run, is that it has an incredible debugging system and alerts from the programmer to increase the ability to write bug-free programmes. Especially when compared to other programming languages, such as C and C++. Meaning that it is a good choice in game design, as it can find bugs easier than other programmes, which is a very high priority issue within the development of video games.

Another strength is that C# has, in terms of creating of a User Interface, is that it creates a very early feel and inspection of the Application way before it’s completion. Essentially, this method of creating the UI gives the developer a very good idea on how their application would like like, making it possible to point out any personal and visual improvements.

However, the weaknesses that C# provides Hard to learn for beginners. To elaborate, C# isn't a very friendly language for inexperienced developers. For instance, a programmer may be able to comprehend the language but that doesn't essentially mean they wont make mistakes. As a very experienced expert is needed to find the issue/error. The reason is due to the libraries being incredibly vast, as mentioned previously, which would take multiple years of experience to have knowledge of the said library.

Another weakness is that using C# for the creation of a UI, is that the application will contain a significant amount of non-maintainable code. Essentially, while the creation of the Ui through this method is advantageous in the fact the developer has an early feel for the application, there is a disadvantage of the application not being able to be fully modified due to this feature. Hence the fact that it is called non-maintainable code, essentially reducing the ability of fully customising the programme.

Discuss at least two advantages and two disadvantages of using classes, inheritance and polymorphism when creating your game

The use of the essential object oriented techniques with the use of classes, such as inheritance and polymorphism, can have its own advantages and disadvantages in video game design. Due to the method in which it uses classes to store data, but how it also has the ability of affecting certain aspects as well.

One advantage to object oriented paradigm and its use of classes, is that the language paradigm offers significant reusability of classes. Essentially, the programmer has the ability of reusing the classes, especially ones which were already created (predefined classes), rather than constantly repeating code.

Another advantage is the fact that everything is treated as objects. Therefore, this gives the ability for the programmer to model what would essentially be a real-world concept by purely using object oriented programming. Essentially, the OOP paradigm offers an almost limitless ability of creating any game within the creator's imagination.

Although, while the use of object oriented is that it can become very large in size even for simple task and can cause problems because of it. Essentially, the size of programmes developed with the object oriented paradigm, such as the use of classes and inheritance/polymorphism, are significantly larger than other paradigms (i.e the procedural paradigm approach). Additionally, since the programme is larger in size, it means that more instructions are required to be executed. Which, in turn results in a significant impact of performance of the user's personal computer, due to the slower execution of programmes.

Another disadvantage is that it is hard to learn for beginners. To elaborate, due to the fact that a majority of the OOP paradigm is treated as objects, the programmers will need a requirement of design skills, programming skills and experience and a different method of thinking, especially in terms of objects. Due to this fact, the programmer should have a well thought out plan before designing an application, which is very important for an object oriented approach.

Task 5 - Maintenance *(Unit 22 D2)*

Task description - *Explain (with a good example from your game for each) what techniques you have used to:*

- *Reduce the total code in your game.*
- *Simplify the code in the main (form) code.*
- *Make it easier to modify / extend your game.*
- *Make your program code as readable and intelligible as possible.*

Reduce the total code in your game

The method in which I have reduced the total amount of code within my application was the extensive use of classes within the class diagram. Which I have used to distribute on all forms, instead of repeating code within all the forms. To elaborate, the reduction of code is important due to the complication of applying once change to some places, would increase the chances of bug developing everywhere or implement the new feature inconsistently.

For example, within my application. I have created a class diagram consisting of all the weapons which will be used throughout the entire game. This class diagram has reduced the amount of code due to the fact I would only have to refer the data within these classes through the “gamedata” class I have created, in order to transfer data within forms.

Essentially, the duplication of gode isn’t necessarily a good thing within an application. Due to the fact that changing it requires making the same change to multiple places in your program. Which is evidently problematic. Meaning that duplicate code is a very undesirable aspect and having duplicate code in the program is the perfect setting for bugs.

Simplify the code in the main (form) code

With similar techniques used above, I have also simplified the code within the form code. A simple programme has the ability to be written in multiple ways, especially if given to a variety of developers. Due to this, the best practice for writing programme code is to write simple, as it can help standardize functions and help reduce overall bugs and future debugging.

For example, as mentioned above, the use of classes also made it significantly easier to read the code within the actual form within the application. Due to the fact I have such vital variables within my classes, I only have to reference them within the form code under the event handlers they are assigned to. Increasing the simplicity of the code significantly, and making aspects such as identifying and debugging easier.

Essentially, this is useful because it gives the code the ability of getting to the point, reducing the amount of confusing and unnecessary clutter, which could cause a wide amount of confusion. Which therefore increase the quality of the code, because it increases productivity while decreasing the amount of time taken for said productivity.

Make it easier to modify / extend your game

With such heavy uses of classes within my application, It gave me the ability to create simple code which can always be added and modified. Essentially, the use of class can give the ability to extend and modify the code without actually changing the form code.

For instance, due to the fact that the code is simple, it makes it significantly easier to modify. Especially with the use of polymorphism within each class. For instance, within my programme, I have used polymorphism within each weapon class to have different numerical values, which is determined as the damage for those said weapons. While also having the ability to add many more attributes, which could be transferred due to the gamedata class I have created.

Therefore, if none of the files were named after what file type, they were changing script for the objects or changing sprites would be incredibly time draining and if another user is working on the game and they can't find the direct name of the object, sprite background or sound it would be near impossible for them to do anything with the game.

Make your program code as readable and intelligible as possible

Due to the vast simplicity, with the addition of the naming of variable and objects. Modularization is where you organize information into sections so it is easier to read and locate issues.

This will be done before the code is even been implemented, this will take place before the code is even made; this is basically a template which people can follow so they know where the code must be, and how it should be laid out. The advantage this brings is that now the code is in different sections you can give it to different people so everyone is working on a single project.

For example, the way I have made the code within my programme much more readable and intelligible as possible to humans, while also having the correct syntax for the application. Essentially, I used meaningful naming conventions, which is essentially using short and simple to understand abbreviations, such as "btn" and "lbl". The reason that I used "btn", instead of the whole word "Button", makes it much more intelligible for the human to read and also makes it easier to structure the application.

Essentially, when creating a game application, especially if it is a large game, it is essential to be able to find and comprehend all the variables and objects within the programme, in such a manner to be identified quickly and debugged quickly if necessary. With the use of simple, easy to remember words, such as a longsword button would be identified as "btn_longsword", and any other weapon such as the battle axe in the form of "btn_battleAxe".

Conclusion

In conclusion, the application I have created fits the original requirements that was assigned within the vocational context. Featuring event driven programming, object oriented programming and it is a functioning video game. Albeit it does have its flaws, it doesn't mean it cannot be improved. With the programme being coded to simplicity and with the good use of modularization, means it is significantly easier to upgrade and debug.

Overall, the game that I have designed is relatively simple in code, but it means that it has very good quality rather than it being incompetent to run a video game. Due to the fact that the game runs perfectly fine, with the right amount of code necessary to make it run efficiently. However, while it's simplicity does give the programme the quality, the programme itself is relatively simple in design and technologically in general. However, even with it's flaws, it proves to be a very capable game with both uses event driven and object oriented paradigms.